CHALLENGE DATACITY · EFREI · FILIÈRE DÉVELOPPEMENT WEB

TrajectAI

Thématique MOVECITY — « Une ville qui fluidifie les mobilités quotidiennes »

Documentation d'architecture — version finale

Livrable J4 — Prototype finalisé et architecture technique

État du prototype · Architecture · API · Données · Moteur d'optimisation · IA · Qualité · Déploiement

Élément	Détail
Projet	TrajectAI — optimisation explicable des trajets multi-étapes
Filière	Développement web
Équipe projet	Kavisha RUBAKANTHAN · Nono Yvana NOUTCHANG · Luân TA · Ywen MASSON · Noussirath BELLO · Cyril DABERNARD
Date	6 octobre 2026 — consolidation du Jalon 1
Prototype en ligne	https://trajectai-paris.vercel.app
Version	Architecture v2 (fait suite à l'architecture v1 du livrable J3)

Sommaire

Introduction 3
Ce qui a changé depuis la v1 (J3) 3
1. État du prototype finalisé 4
1.1 Fonctionnalités livrées 4
1.2 Aperçu du prototype 5
2. Architecture technique v2 6
2.1 Vue d'ensemble 6
2.2 Composants et technologies 6
2.3 Organisation du code 7
3. Référence de l'API 8
3.1 Exemple : réponse de POST /optimize (extrait) 8
4. Données 9
4.1 Ce qui est stocké, et où 9
4.2 Modèle de données Supabase 9
4.3 Données ouvertes et services externes 9
5. Moteur d'optimisation explicable 11
5.1 Déroulé d'un calcul 11
5.2 Simulation d'une étape 11
5.3 Score 11
6. Intelligence artificielle 12
6.1 Assistant Traject 12
6.2 Vision « Où suis-je ? » 12
6.3 Voix 12
7. Qualité et performances 13
7.1 Tests automatisés 13
7.2 Performances mesurées en ligne 13
8. Déploiement et exploitation 14
8.1 Installation locale 14
8.2 Mise en production 14
9. Limites connues et prochaines étapes 15
Backlog développement — Partie 2 15

Introduction

Ce document est la **documentation d’architecture finale** de TrajectAI pour le Jalon 1. Il fait suite à l’architecture technique v1 présentée au J3 : il en reprend les principes, les met à jour avec les évolutions du prototype et détaille ce dont une équipe a besoin pour comprendre, faire tourner et faire évoluer la plateforme.

Le principe directeur reste inchangé : **le citoyen liste, l’IA optimise, la ville informe, l’usager contrôle.** Côté technique, il se traduit par une règle forte : **le serveur qui calcule les trajets ne possède aucune base de données.** Les trajets ne peuvent donc pas être conservés, même par erreur.

Ce qui a changé depuis la v1 (J3)

Évolution	Impact sur l’architecture
Orbe vocale et dictée de la journée	L’assistant peut créer des étapes à partir d’une phrase (nouvelle action « addStops »)
« Où suis-je ? » par la caméra	Nouvelle route POST /api/vision/locate et modèle de vision open source
Conseil selon la météo	Nouveau service externe Open-Meteo ; la pluie pénalise le vélo et la marche dans le calcul
Recherche à proximité	Les lieux sont cherchés et triés autour du point de départ
Liste de courses	Le temps passé au supermarché dépend du nombre d’articles
Page « Mes trajets »	Historique facultatif, stocké uniquement dans le navigateur de l’usager

1. État du prototype finalisé

Le prototype dépasse le stade du « squelette » attendu au Jalon 1 : il est **fonctionnel, déployé en ligne et couvre l'intégralité des fonctionnalités indispensables du MVP**.

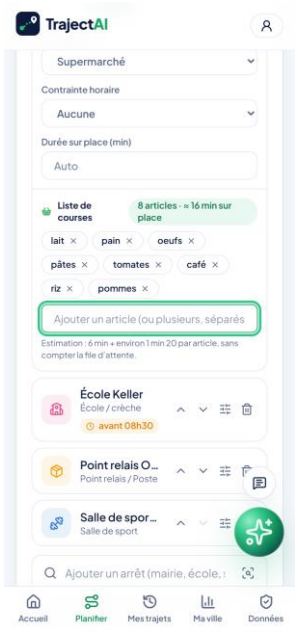
1.1 Fonctionnalités livrées

Pilier	Fonctionnalité	État
Lister	Saisie de 2 à 6 étapes, type de lieu détecté automatiquement, contraintes horaires, liste de courses	Livré
Lister	Dictée de la journée à la voix ou à l'écrit (l'IA crée les étapes)	Livré
Lister	Recherche à proximité, adresses récentes, « Où suis-je ? » par la caméra	Livré
Optimiser	Ordre optimal selon la priorité (temps, coût, CO_2), trafic réel, affluence, horaires, météo	Livré
Informer	Carte, heure de chaque étape, explication par étape, gain par rapport à l'ordre saisi	Livré
Informer	Météo heure par heure et mode de transport conseillé	Livré
Contrôler	Changement de priorité ou de mode, réorganisation manuelle, recalcul instantané	Livré
Contrôler	Zéro historique serveur, effacement du trajet, historique local facultatif	Livré
Guidage	Guidage vocal avec détection d'arrivée (GPS local) et mode démonstration	Livré
Dashboard	4 visualisations de données ouvertes (trafic, saturation, CO_2, Vélib')	Livré
Compte	Inscription, connexion, préférences, favoris, export et suppression du compte	Livré
Responsive	Toutes les pages vérifiées sur un écran de 390 px, sans défilement horizontal	Livré

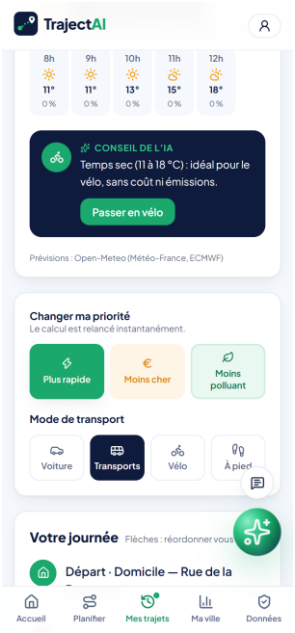
1.2 Aperçu du prototype



Accueil



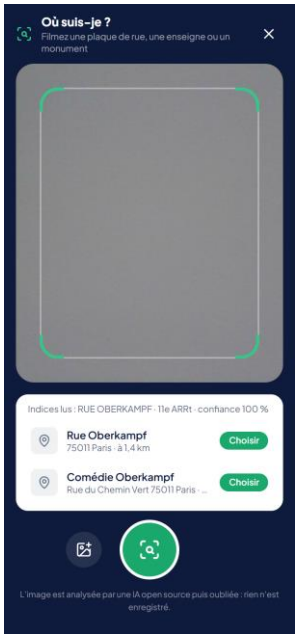
Planifier + liste de courses



Météo et conseil de l'IA



Mes trajets



« Où suis-je ? »



Orbe vocale Traject

Figure 1 — Captures du prototype en ligne, sur téléphone

2. Architecture technique v2

2.1 Vue d'ensemble

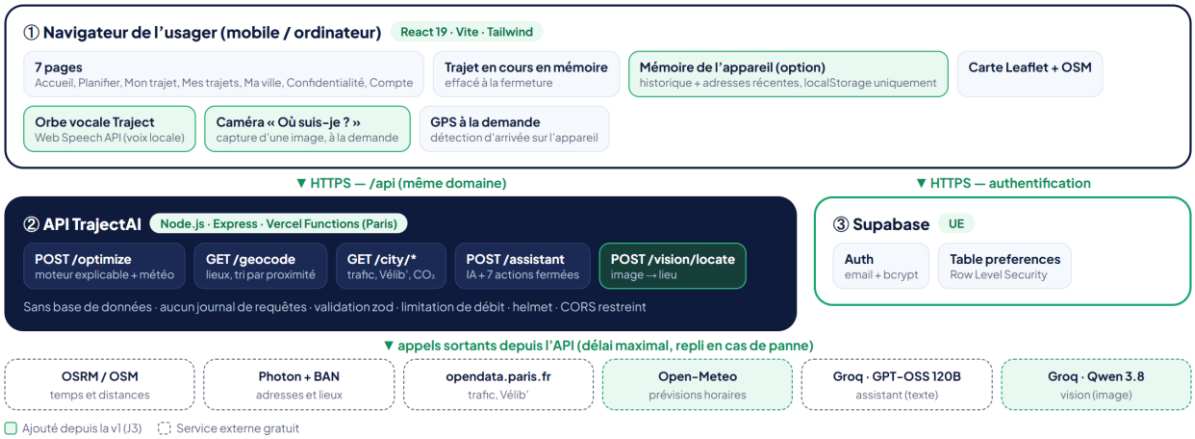


Figure 2 — Architecture de TrajectAI (en vert : ajouts depuis la v1)

1. Le navigateur (React + Tailwind)

L'utilisateur arrive sur une interface responsive pensée mobile d'abord, organisée en sept pages : Accueil, Planifier, Mon trajet, Mes trajets, Ma ville, Confidentialité et Compte. Le trajet calculé reste en mémoire le temps de la session et disparaît à la fermeture de l'onglet. **Seule exception, à la demande de l'usager** : l'historique et les adresses récentes peuvent être conservés dans le navigateur de l'appareil, jamais sur nos serveurs, et s'effacent en un clic. La carte s'appuie sur Leaflet et OpenStreetMap. La voix et la géolocalisation sont traitées localement dans le navigateur (Web Speech API, géolocalisation à la demande).

2. L'API TrajectAI (Node.js / Express, déployée sur Vercel, région Paris)

Cette API n'a pas de base de données, ne journalise aucune requête, valide les entrées avec zod et applique une limitation du nombre de requêtes ainsi que les en-têtes de sécurité de helmet. C'est un choix d'architecture qui élimine une bonne partie des risques classiques (fuite de journaux, conservation de données personnelles) avant même d'avoir à les corriger. Elle orchestre les services externes : calcul d'itinéraires, recherche de lieux, données ouvertes, météo et modèles d'IA.

3. Supabase (hébergé dans l'UE)

Rôle volontairement restreint : authentification par email et mot de passe (haché avec bcrypt) et **une seule table**, preferences, protégée par Row Level Security. Chaque utilisateur ne peut lire ou écrire que ses propres préférences. Aucune donnée de trajet n'y transite.

2.2 Composants et technologies

Composant	Technologie	Rôle et justification
Interface web	React 19, Vite, Tailwind CSS 4	Site responsive, mobile d'abord ; pages chargées à la demande pour un démarrage rapide.
Carte	Leaflet + OpenStreetMap	Affichage de l'itinéraire ; solution libre, sans clé payante.

Composant	Technologie	Rôle et justification
Graphiques	Recharts	Visualisations interactives du dashboard, avec infobulles et tableaux de données.
Voix	Web Speech API du navigateur	Reconnaissance et synthèse vocales gratuites, sans serveur supplémentaire.
API	Node.js + Express	Même langage que l’interface ; sans état et sans base de données.
Moteur d’optimisation	Algorithme développé par l’équipe	Teste tous les ordres possibles (jusqu’à 720) et justifie chaque choix par des chiffres.
Assistant IA	GPT-OSS 120B (open source) via Groq	Comprend les demandes, répond et déclenche des actions ; moteur de règles local en secours.
Vision	Qwen 3.8 (open source) via Groq	Lit plaques de rue et enseignes sur une image pour retrouver le lieu.
Comptes	Supabase (UE)	Authentification sécurisée et stockage des seules préférences.
Hébergement	Vercel (fonctions serverless, région Paris)	Interface et API sous la même adresse ; déploiement en une commande.

2.3 Organisation du code

Le dépôt est séparé en trois parties : l’interface (client), l’API (server) et la base (supabase). Environ 5 700 lignes de code au total.

```
TrajectAI/
├── api/index.js              Point d’entrée Vercel : l’API Express en fonction serverless
├── vercel.json               Build, région Paris (cdg1), redirections /api et SPA
├── client/src/
│   ├── pages/                Landing, Planner, Result, History, Dashboard, Privacy, Auth, Account
│   ├── components/           Assistant (orbe + chat), CameraLocate, PlaceSearch, StopRow,
│   │                         TripMap, WeatherCard, GuidancePanel, Layout…
│   ├── context/              TripContext (trajet en mémoire), AuthContext (Supabase)
│   └── lib/                  api, memory (historique local), speech (voix), supabase, constants
├── server/src/
│   ├── app.js                Sécurité, limites de débit, montage des routes
│   ├── routes/               optimize, geocode, city, assistant, vision
│   └── lib/                  optimizer (moteur), profiles (paramètres), routing (OSRM),
│                             opendata (Ville de Paris), weather (Open-Meteo)
├── server/data/              Instantanés de secours des données ouvertes
├── server/test/              Tests unitaires du moteur
└── supabase/schema.sql       Table preferences, règles RLS, suppression de compte
```

3. Référence de l’API

Toutes les routes sont servies sous **/api**, sur le même domaine que le site. Les réponses contenant des données personnelles portent l’en-tête « Cache-Control: no-store ».

Méthode et route	Rôle	Entrée principale	Limite / min
GET /health	Vérifier que l’API répond	—	—
POST /optimize	Calculer l’ordre optimal, les options par mode et la météo	Départ, heure, mode, priorité, 2 à 6 étapes (contrainte, durée, liste de courses), ordre imposé facultatif	40
GET /geocode	Rechercher un lieu, trié par proximité	Texte ; position facultative	120
GET /geocode/reverse	Adresse d’une position GPS	Latitude, longitude	120
GET /city/traffic	Trafic par jour et par heure	—	60
GET /city/velib	Vélib’ disponibles par arrondissement	—	60
GET /city/co2	Facteurs d’émission par mode	—	60
GET /assistant/status	Modèle d’IA actif	—	20
POST /assistant	Réponse de Traject et action à exécuter	Messages et contexte (trajet, météo, brouillon)	20
POST /vision/locate	Retrouver un lieu à partir d’une image	Image JPEG ≤ 1,5 Mo, position facultative	10

3.1 Exemple : réponse de POST /optimize (extrait)

```
{
  "plan": {
    "steps": [{ "label": "École Keller", "arrival": "8h16",
                "explanation": "À atteindre avant 08h30 : placé en 1re position…",
                "queueMin": 0, "items": 0, "weather": { "temp": 14, "pop": 0 } }, …],
    "totals": { "durationMin": 240, "cost": 12.3, "co2": 0.3, "endTime": "12h00" }
  },
  "gains":   { "minutes": 20, "euros": 0, "co2": 0.1 },
  "options": [{ "mode": "bike", "totals": { … } }, …],
  "weather": { "summary": "Couvert, pas de pluie prévue",
               "recommended": { "mode": "bike", "reason": "Temps sec : idéal pour le vélo…" } },
  "geometry": [[48.853, 2.381], …]
}
```

4. Données

4.1 Ce qui est stocké, et où

Donnée	Emplacement	Durée
Trajet en cours	Mémoire de l'onglet (React)	Jusqu'à la fermeture ou « Effacer »
Étapes envoyées au calcul	API (traitement uniquement)	Aucune conservation
Email et mot de passe haché	Supabase, table auth.users	Tant que le compte existe
Préférences et lieux favoris	Supabase, table preferences	Tant que le compte existe
Historique et adresses récentes (option)	Navigateur de l'usager (localStorage)	Jusqu'à effacement par l'usager
Compteur mensuel d'économies	Navigateur de l'usager	Le mois en cours
Image « Où suis-je ? »	Envoyée au modèle de vision	Aucune conservation

4.2 Modèle de données Supabase

Une seule table applicative. **L'absence volontaire de table de trajets garantit le « zéro historique » par construction.**

Colonne de preferences	Type	Règle
user_id	uuid (clé)	Référence auth.users, suppression en cascade
default_priority	text	time, cost ou co2
default_mode	text	car, transit, bike ou walk
voice_enabled	boolean	Réponses vocales de l'assistant
save_favorites	boolean	Consentement à l'enregistrement des favoris
favorites	jsonb	10 lieux maximum ; vidé automatiquement si le consentement est retiré (trigger)
updated_at	timestamptz	Mis à jour automatiquement

Sécurité de la base : 4 règles Row Level Security (lecture, création, modification, suppression limitées à sa propre ligne) et une fonction delete_my_account() réservée aux utilisateurs connectés pour le droit à l'effacement.

4.3 Données ouvertes et services externes

Source	Usage	Résilience
Ville de Paris — comptages routiers	Facteur de congestion heure par heure	Cache 24 h + instantané local
Vélib' Métropole — temps réel	Dashboard	Cache 5 min + instantané local
ADEME — Impact CO_2	Émissions par mode	Valeurs intégrées au code

Source	Usage	Résilience
OSRM / OpenStreetMap	Temps et distances, tracé	Repli : estimation à vol d’oiseau
Photon + Base Adresse Nationale	Recherche de lieux	Repli Photon → BAN
Open-Meteo	Prévisions horaires	Cache 30 min ; sans météo, le calcul continue
Groq (GPT-OSS 120B, Qwen 3.8)	Assistant et vision	Repli : moteur de règles local pour l’assistant

5. Moteur d'optimisation explicable

Le moteur est le cœur de l'« IA » de TrajectAI. C'est un **algorithme d'optimisation exhaustif et explicable** : il ne devine pas, il compare toutes les possibilités et peut justifier chaque choix par des chiffres.

5.1 Déroulé d'un calcul

1. Validation stricte de la requête (types, bornes, 2 à 6 étapes).
2. Récupération en parallèle du trafic, de la météo et des matrices de temps et distances pour les 4 modes.
3. Pour chaque ordre possible (6! = 720 au maximum), simulation de la journée minute par minute.
4. Calcul d'un score selon la priorité ; l'ordre au score le plus bas est retenu.
5. Rédaction d'une explication par étape, du gain par rapport à l'ordre saisi, d'un conseil d'heure de départ et du mode recommandé selon la météo.

5.2 Simulation d'une étape

```
arrivée   = départ précédent + trajet × facteur de trafic(jour, heure)
            (+ 5 min de recherche de stationnement en voiture)
début     = max(arrivée, ouverture du lieu, contrainte « pas avant »)
attente   = file maximale du lieu × affluence(heure)
durée     = durée saisie, sinon 6 min + 1,3 min × articles (courses), sinon durée type
violation = retard sur une contrainte « avant » ou fermeture du lieu
```

5.3 Score

```
score = a × durée totale (min) + b × coût (€) + c × CO₂ (kg)
      + 50 × minutes de contraintes non respectées
      + 1,5 × minutes à vélo ou à pied sous la pluie

Priorité « temps »  : a = 1     b = 1    c = 1
Priorité « coût »   : a = 0,15  b = 10   c = 1
Priorité « CO₂ »    : a = 0,15  b = 1    c = 40
```

Les paramètres (coûts, émissions ADEME, horaires et profils d'affluence par type de lieu) sont regroupés dans un seul fichier, server/src/lib/profiles.js, et chacun est sourcé ou signalé comme hypothèse. Les profils d'affluence sont modélisés, faute de données ouvertes sur la fréquentation des guichets.

6. Intelligence artificielle

6.1 Assistant Traject

L'assistant utilise **GPT-OSS 120B**, un modèle open source, via l'API gratuite de Groq. Le serveur lui transmet un contexte limité (résumé du trajet, météo, étapes en préparation) et lui impose de répondre au format JSON : une réponse à lire à voix haute et **une action choisie dans une liste fermée**.

Action	Effet dans l'application
setPriority	Change la priorité (temps, coût, CO_2) et recalcule
setMode	Change le mode de transport et recalcule
optimize	Lance le calcul
startGuidance	Démarre le guidage vocal
nextStep	Passe à l'étape suivante (« je suis arrivé »)
openPage	Ouvre une page de l'application
addStops	Crée les étapes dictées : chaque lieu est recherché près du départ, puis l'ordre est calculé

Toute action hors de cette liste est ignorée, et les étapes dictées sont nettoyées (6 au maximum, catégories et heures vérifiées). L'IA ne peut donc pas déclencher d'action imprévue. Sans clé d'IA, un moteur de règles local répond aux questions les plus courantes.

6.2 Vision « Où suis-je ? »

L'image capturée par la caméra est réduite à 1 024 pixels, puis envoyée à **Qwen 3.8**, un modèle de vision open source. Il renvoie le lieu le plus précis qu'il peut lire (plaque de rue, enseigne, monument), qui est ensuite recherché près de l'usager. L'image n'est ni stockée ni journalisée.

6.3 Voix

La reconnaissance et la synthèse vocales utilisent la Web Speech API du navigateur. L'orbe vocale enchaîne quatre états visibles : en attente, écoute, réflexion, réponse.

7. Qualité et performances

7.1 Tests automatisés

Test unitaire du moteur	Résultat
La contrainte « école avant 8h30 » est respectée	Réussi
L’ordre optimal n’est jamais pire que l’ordre saisi	Réussi
Une contrainte impossible est signalée	Réussi
Une mairie fermée le dimanche est pénalisée	Réussi
Chaque étape reçoit une explication	Réussi
Le trafic réel ralentit la voiture en heure de pointe	Réussi
La pluie pénalise le vélo mais pas la voiture	Réussi
La liste de courses allonge le temps au supermarché	Réussi

En complément, des parcours complets sont rejoués automatiquement dans un navigateur Chrome, au format téléphone, sur la version en ligne : planification, optimisation, météo, historique, recherche à proximité, « Où suis-je ? » et dictée à l’orbe. Dernier passage : **aucune erreur, aucun débordement horizontal**.

7.2 Performances mesurées en ligne

Opération	Temps mesuré
Calcul d’un trajet optimisé (serveur déjà actif)	0,1 à 0,3 s
Premier calcul après une période d’inactivité (réveil du serveur)	environ 8 s
Réponse de l’assistant IA	environ 1 s
Recherche de lieu à proximité	0,5 à 3 s
« Où suis-je ? » (analyse de l’image + recherche du lieu)	environ 3 s

8. Déploiement et exploitation

8.1 Installation locale

```
npm run setup     # installe la racine, le serveur et le client
npm run dev       # API sur le port 3001 + site sur http://localhost:5173
npm test          # tests unitaires du moteur
npm run build     # version de production du site
```

8.2 Mise en production

Un seul projet Vercel héberge le site (fichiers statiques) et l’API (fonction serverless en région Paris). La commande « npx vercel deploy --prod » publie la version de production, servie à l’adresse https://trajectai-paris.vercel.app.

Variable d’environnement	Rôle
AI_API_KEY	Clé de l’API d’IA (Groq), côté serveur uniquement
AI_MODEL / AI_BASE_URL	Modèle et fournisseur de l’assistant (Ollama possible en local)
AI_VISION_MODEL	Modèle de vision (facultatif, Qwen 3.8 par défaut)
VITE_SUPABASE_URL / VITE_SUPABASE_ANON_KEY	Connexion du site à Supabase (clé publique)
CORS_ORIGIN	Origines autorisées pour l’API

Secrets
Aucune clé secrète n’est présente dans le code ni dans le site : elles sont stockées dans les variables d’environnement de Vercel et dans des fichiers .env exclus de Git.

9. Limites connues et prochaines étapes

Limite	Piste d’amélioration
Affluence des lieux modélisée, pas mesurée	Brancher des données réelles de fréquentation si la ville les ouvre, ou les signalements des citoyens
Transports en commun estimés (distance et vitesse moyenne)	Intégrer un calculateur d’itinéraires en transports (données GTFS d’Île-de-France Mobilités)
Premier calcul lent après inactivité	Garder le serveur actif par un appel régulier ou une offre d’hébergement sans mise en veille
Voix limitée à Chrome et Edge	Repli texte déjà en place ; ajouter une transcription côté serveur (Whisper, open source)
Services d’IA hébergés hors UE	Proposer un hébergement européen ou un modèle local (Ollama)

Backlog développement — Partie 2

- Intégrer les corrections issues de l’audit de sécurité v1.
- Ajouter la participation citoyenne : signalement anonyme de l’affluence en temps réel.
- Export de la journée vers l’agenda et option d’accessibilité (stations avec ascenseur).
- Tests utilisateurs avec la filière marketing, puis ajustements de l’interface.